

Hiwonder Magnetic Encoder Bus Servo Communication Protocol Document

Hiwonder Magnetic Encoder Bus Servo Communication Protocol Document

- 1.0 Overview
 - 1.1 Command Packet
 - 1.2 Response Packet
 - 1.3 Instruction Types
 - 1.3.1 Query Status Command PING
 - 1.3.2 Read Command READ DATA
 - 1.3.3 Write Command WRITE DATA
 - 1.3.4 Asynchronous Write Command REG WRITE
 - 1.3.5 Execute Asynchronous Write Command ACTION
 - 1.3.6 Synchronous Write Command SYNC WRITE
 - 1.3.7 Synchronous Read Command SYNC READ
 - 1.3.8 Reset Command RESET
 - 1.4 Operation Instruction

1.0 Overview

- This communication protocol applies to the **HX-30HM magnetic encoder** bus servo. The servo uses an ARM 32-bit microcontroller as the main control chip, and its position sensing utilizes a **360-degree, 12-bit precision** magnetic sensing angle solution.
- Multiple servos are allowed in a single bus control network, so **each servo is assigned a unique ID number within the network**. The control instructions sent by the controller contain the ID information, and **only the servo with the matching ID can fully receive the instruction and return a response**.
- The communication method is **serial asynchronous**, controlled uniformly through the UART asynchronous serial interface. **The communication speed is 1 Mbps**, with each data frame consisting of 1 start bit, 8 data bits, and 1 stop bit, with no parity bit, totaling 10 bits.
- For detailed control content, refer to the HX-35HM memory table.

1.1 Command Packet

Command packet format:

Header	ID Number	Data Length	Instruction	Parameters	Checksum
0xFF 0xFF	ID	Length	Instruction	Parameter 1 ...Parameter N	Check Sum

- **Header:** Receiving two consecutive 0xFF bytes indicates that a data packet has arrived.

- **ID Number:** Each servo has a unique ID number. **The ID number range is from 0 to 253**, which corresponds to the hexadecimal range of 0x00 to 0xFD. **ID number 254 (0xFE) is reserved for broadcasting.** When the controller sends an ID 254 (0xFE), all servos will receive the instruction. However, except for the PING instruction, no other instructions will return a response. **Broadcast PING instructions cannot be used when multiple servos are connected on the bus.**
- **Data Length:** Equal to the number of parameters to be sent (N) plus 2, i.e., **N+2**.
- **Instruction:** The operation function code of the data packet. For details, refer to [1.3 Instruction Types](#).
- **Parameters:** Additional control information required by the instruction. The parameters support a maximum of two-byte values to represent a memory value. The byte order is as specified in the HX-35HM memory table.
- **Checksum:** The calculation method is as follows:

$$\text{Check Sum} = \sim (\text{ID} + \text{Length} + \text{Instruction} + \text{Parameter 1} + \dots + \text{Parameter N})$$

If the sum inside the parentheses exceeds 255, only the least significant byte is taken. "~" denotes the bitwise NOT operation.

1.2 Response Packet

The response packet is the reply from the servo to the controller. The format of the response packet is as follows:

Header	ID Number	Data Length	Current Status	Parameters	Checksum
0xFF 0xFF	ID	Length	ERROR	Parameter 1 ...Parameter N	Check Sum

- The returned response packet includes the servo's current status, indicated by the **ERROR** byte. If the servo's current operating status is abnormal, it will be reflected in this byte. For detailed meanings of each status, refer to the HX-35HM memory table. **If ERROR is 0, there are no error messages from the servo.**
- If the instruction is a read instruction (**READ DATA**), then **Parameter 1 ... Parameter N** contains the read information.

1.3 Instruction Types

The available commands for the bus intelligent servo communication protocol are as follows:

Instruction	Function	Value	Parameter Length
PING (Query)	Query working status	0x01	0
READ DATA(Read)	Query characters in the control table	0x02	2
WRITE DATA (Write)	Write characters into the control table	0x03	Greater than or equal to 1

Instruction	Function	Value	Parameter Length
REGWRITE DATA (Asynchronous Write)	Similar to WRITE DATA, but the control characters do not take effect immediately after being written. The action will occur when the ACTION command is received	0x04	Not less than 2
ACTION (Execute Asynchronous Write)	Trigger the action for REG WRITE	0x05	0
SYNCREAD DATA (Synchronous Read)	Used to query multiple servos simultaneously	0x82	Greater than or equal to 3
SYNCWRITE DATA (Synchronous Write)	Used to control multiple servos simultaneously	0x83	Not less than 2
RESET (Reset)	Reset the control table to factory default values	0x06	0

1.3.1 Query Status Command PING

- **Function:** Read the servo's working status
- **Length:** 0x02
- **Instruction:** 0x01
- **Parameters:** None

Example: Read the working status of the servo with ID number 1

- Instruction Frame (Hexadecimal): **FF FF 01 02 01 FB**

Header	ID	Valid Data Length	Instruction	Checksum
0xFF 0xFF	0x01	0x02	0x01	0xFB

- Return Frame (Hexadecimal): **FF FF 01 02 00 FC**

Header	ID	Valid Data Length	Working status	Checksum
0xFF 0xFF	0x01	0x02	0x00	0xFC

1.3.2 Read Command READ DATA

- **Function:** Read data from the servo's memory control table
- **Length:** 0x04
- **Instruction:** 0x02
- **Parameter 1:** Starting address of the data to be read
- **Parameter 2:** Length of the data to be read

Example: Read the current position of the servo with ID 1 (low byte first, high byte second).

Read two bytes from address **0x38** in the control table.

- Instruction Frame (Hexadecimal): **FF FF 01 04 02 38 02 BE**

Header	ID	Valid Data Length	Instruction	Parameters	Checksum
0xFF 0xFF	0x01	0x04	0x02	0x38 0x02	0xBE

- Returned Data Frame (Hexadecimal): **FF FF 01 04 00 18 05 DD**

Header	ID	Valid Data Length	Working status	Parameters	Checksum
0xFF 0xFF	0x01	0x04	0x00	0x18 0x05	0xDD

Read two bytes of data: Low byte: 0x18; High byte: 0x05. The two bytes combined form a 16-bit data value: 0x0518. The current position in decimal is 1304.

1.3.3 Write Command WRITE DATA

- **Function:** Write data to the servo's memory control table
- **ID:** The ID number of the servo to be controlled
- **Length:** N + 3 (where N is the data length)
- **Instruction:** 0x03
- **Parameter 1:** Starting address of the data to be written
- **Parameter 2:** First data to be written
- **Parameter 3:** Second data to be written
- **Parameter N + 1:** The N-th data

Example: Set an arbitrary ID to 1.

The address in the control table that stores the ID number is 5, so write 1 to address 5. Send the instruction packet with the ID using the broadcast ID (0xFE).

- Instruction Frame (Hexadecimal): **FF FF FE 04 03 05 01 F4**

Header	ID	Valid Data Length	Instruction	Parameters	Checksum
0xFF 0xFF	0xFE	0x04	0x03	0x05 0x01	0xF4

Since the instruction is sent using the broadcast ID, no data will be returned. Additionally, the memory table NVS has a protection lock switch. Before modifying the ID, it needs to be turned off (set to 0), otherwise, the example ID will not be saved after power off. For detailed operations, please refer to the memory table or the operation manual.

Example: Control the servo with ID 1 to rotate to the position 2048 at a speed of 1000 per second.

The starting address for the target position in the control table is 0x2A, so write six consecutive bytes of data starting at address 0x2A, as follows:

Position data: **0x0800 (2048)**

Time data: **0x0000 (0)**

Speed data: **0x03E8 (1000)**

Send the instruction packet with a non-broadcast ID (0xFE), so after the instruction is received, the servo will return a status packet.

- Instruction Frame (Hexadecimal): **FF FF 01 09 03 2A 00 08 00 00 E8 03 D5**

Header	ID	Valid Data Length	Instruction	Parameters	Checksum
0xFF 0xFF	0x01	0x09	0x03	0x2A 0x00 0x08 0x00 0x00 0xE8 0x03	0xD5

- Returned Data Frame (Hexadecimal): **FF FF 01 02 00 FC**

Header	ID	Valid Data Length	Working status	Checksum
0xFF 0xFF	0x01	0x02	0x00	0xFC

The returned working status is **0**, indicating that the servo has correctly received the instruction and has started executing it.

1.3.4 Asynchronous Write Command REG WRITE

The REG WRITE command is similar to the WRITE DATA command, but the execution time is different. When the REG WRITE instruction frame is received, the data is stored in the buffer for later use, and the Registered Instruction register is set to 1. Once the ACTION command is received, the stored instruction will be executed.

- **Function:** Write data to the servo's memory control table
- **Length:** N + 3 (where N is the **parameter length**)
- **Instruction:** 0x04
- **Parameter 1:** Starting address of the data write area
- **Parameter 2:** The first data to be written
- **Parameter 3:** The second data to be written
- **Parameter N + 1:** The N-th data to be written

Example: Control servos from ID 1 to ID 10 to rotate to position 2048 at a speed of 1000 per second. In the following instruction packet, only ID 1 on the bus receives the instruction and returns, while other ID numbers do not return on the bus.

ID 1 Asynchronous Write Instruction Packet: FF FF 01 09 04 2A 00 08 00 00 E8 03 D4
ID 1 Response Packet: FF FF 01 02 00 FC
ID 2 Asynchronous Write Instruction Packet: FF FF 02 09 04 2A 00 08 00 00 E8 03 D3

ID 1 Asynchronous Write Instruction Packet: FF FF 01 09 04 2A 00 08 00 00 E8 03 D4
ID 3 Asynchronous Write Instruction Packet: FF FF 03 09 04 2A 00 08 00 00 E8 03 D2
ID 4 Asynchronous Write Instruction Packet: FF FF 04 09 04 2A 00 08 00 00 E8 03 D1
ID 5 Asynchronous Write Instruction Packet: FF FF 05 09 04 2A 00 08 00 00 E8 03 D0
ID 6 Asynchronous Write Instruction Packet: FF FF 06 09 04 2A 00 08 00 00 E8 03 CF
ID 7 Asynchronous Write Instruction Packet: FF FF 07 09 04 2A 00 08 00 00 E8 03 CE
ID 8 Asynchronous Write Instruction Packet: FF FF 08 09 04 2A 00 08 00 00 E8 03 CD
ID 9 Asynchronous Write Instruction Packet: FF FF 09 09 04 2A 00 08 00 00 E8 03 CC
ID 10 Asynchronous Write Instruction Packet: FF FF 0A 09 04 2A 00 08 00 00 E8 03 CB

1.3.5 Execute Asynchronous Write Command ACTION

When controlling multiple servos, the ACTION command allows all servos from the first to the last to execute their respective actions simultaneously, without any delay in between.

When sending the ACTION command to multiple servos, the broadcast ID (0xFE) must be used. Therefore, no data frame will be returned after sending this instruction.

- **Function:** Trigger the **REG WRITE** instruction
- **Length:** 0x02
- **Instruction:** 0x05
- **Parameters:** None

Example: After sending the asynchronous write instructions to control servos from ID 1 to ID 10 to rotate to position 2048 at a speed of 1000 per second, to execute the asynchronous write instructions,

the following instruction packet needs to be sent: (**FF FF FE 02 05 FA**). After receiving this instruction, all servos on the bus will execute the previously received asynchronous write instructions.

1.3.6 Synchronous Write Command SYNC WRITE

Unlike the REG WRITE + ACTION command, SYNC WRITE has a higher real-time capability. A single SYNC WRITE command can modify the control table content of multiple servos at once, while REG WRITE + ACTION commands are executed step by step. However, when using the SYNC WRITE command, the data length to be written and the starting address for saving data must be the same.

- **Function:** Used to simultaneously control multiple servos
- **ID:** 0xFE
- **Length:** $(L + 1) * N + 4$ (where **L is the data length sent to each servo**, and **N** is the number of servos)
- **Instruction:** 0x83
- **Parameter 1:** Starting address for the data to be written
- **Parameter 2:** Length of the data to be written (L)

- **Parameter 3:** ID number of the first servo
- **Parameter 4:** First data to be written to the first servo
- **Parameter 5:** Second data to be written to the first servo
- ...
- **Parameter L+3:** L-th data to be written to the first servo
- **Parameter L+4:** ID number of the second servo
- **Parameter L+5:** First data to be written to the second servo
- **Parameter L+6:** Second data to be written to the second servo
- ...
- **Parameter 2L+4:** L-th data to be written to the second servo
- ...

Example: Write the position 0x0800, time 0x0000, and speed 0x03E8 (low byte first, high byte second) to the starting address 0x2A for 4 servos with IDs 1 to 4.

- **Instruction Frame (Hexadecimal):** FF FF FE 20 83 2A 06 01 00 08 00 00 E8 03 02 00 08 00 00 E8 03 03 00 08 00 00 E8 03 04 00 08 00 00 E8 03 58

Header	ID	Valid Data Length	Instruction	Parameters	Checksum
0xFF 0xFF	0xFE	0x20	0x83	0x2A 0x06 0x01 0x00 0x08 0x00 0x00 0xE8 0x03 0x02 0x00 0x08 0x00 0x00 0xE8 0x03 0x03 0x00 0x08 0x00 0x00 0xE8 0x03 0x04 0x00 0x08 0x00 0x00 0xE8 0x03	0x58

Since the instruction is sent using the broadcast ID, no data will be returned.

1.3.7 Synchronous Read Command SYNC READ

A single SYNC READ instruction can query the control table content of multiple servos at once. The SYNC READ command specifies the IDs of the servos to be queried, and the servos return response packets in the same order as the IDs in the instruction packet. When using the SYNC READ command, all queried data lengths and starting addresses must be the same (this instruction is available for certain serial bus servos).

- **Function:** Used to simultaneously query multiple servos.
- **ID:** 0xFE
- **Length:** N + 4 (where N is the number of servos)
- **Instruction:** 0x82
- **Parameter 1:** Starting address for the data to be read

- **Parameter 2:** Length of the data to be read
- **Parameter 3:** ID number of the first servo
- **Parameter 4:** ID number of the second servo
- ...
- **Parameter N+2:** ID number of the N-th servo
- ...

Example: Query 2 servos (ID1 to ID2) with the starting address 0x38, retrieving the current position, current speed, current load, current voltage, and current temperature, a total of 8 bytes of data (**low byte first, high byte second**).

- Sync read instruction packet (Hexadecimal): **FF FF FE 06 82 38 08 01 02 36**

Header	ID	Valid Data Length	Instruction	Parameters	Checksum
0xFF 0xFF	0xFE	0x06	0x82	0x38 0x08 0x01 0x02	0x36

- Response Packet (Hexadecimal): **FF FF 01 0A 00 00 08 00 00 00 00 79 1E 55**
- **FF FF 02 0A 00 FF 07 00 00 00 00 77 23 53**

Response Packet Breakdown: ID1's response packet: **FF FF 01 0A 00 00 08 00 00 00 00 79 1E 55**

Header	ID	Valid Data Length	Working status	Parameters	Checksum
0xFF 0xFF	0x01	0x0A	0x00	0x00 0x08 0x00 0x00 0x00 0x00 0x79 0x1E	0x55

ID2's Response Packet: **FF FF 02 0A 00 FF 07 00 00 00 00 77 23 53**

Header	ID	Valid Data Length	Working status	Parameters	Checksum
0xFF 0xFF	0x02	0x0A	0x00	0xFF 0x07 0x00 0x00 0x00 0x00 0x77 0x23	0x53

1.3.8 Reset Command RESET

- **Function:** Reset specific data in the memory control table
- **Length:** 0x02
- **Instruction:** 0x06
- **Parameters:** None

Example: Reset the servo with ID number 0.

- Instruction frame (Hexadecimal): **FF FF 01 02 06 F6**

Header	ID	Valid Data Length	Instruction	Checksum
0xFF 0xFF	0x00	0x02	0x06	0xF7

- Returned Data Frame (Hexadecimal): **FF FF 01 02 00 FC**

Header	ID	Valid Data Length	Working status	Checksum
0xFF 0xFF	0x01	0x02	0x00	0xFC

1.4 Operation Instruction

For certain signed data (such as speed, position calibration parameters), it is important to note the representation during transmission: when the data is less than 0, it is represented in **binary sign-magnitude form**, with the highest bit being the **sign bit** (the highest bit is specified according to the actual situation).

Example: Set the position correction value to -2000 and 2000, with BIT15 as the sign bit. According to the memory table, this data is represented using 2 bytes. The bit positions are represented as follows:

Bit positions when set to -2000:

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
1	0	0	0	0	1	1	1	1	1	0	1	0	0	0	0

Bit positions when set to 2000:

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
0	0	0	0	0	1	1	1	1	1	0	1	0	0	0	0